

IdentifierIDS: A Practical Voltage-Based Intrusion Detection System for Real In-Vehicle Networks

Zhouyan Deng[✉], *Member, IEEE*, Jiajia Liu[✉], *Senior Member, IEEE*, Yijie Xun[✉], *Member, IEEE*,
and Junman Qin[✉], *Graduate Student Member, IEEE*

Abstract—As innovative technologies such as autonomous driving, over-the-air technology, and vehicle-to-everything are widely applied to intelligent connected vehicles, people can gain a more convenient and safer driving experience. Although the application of these technologies facilitates our lives, they also bring a series of vulnerable interfaces (such as 5G, Bluetooth, and WiFi), which pose a significant security threat to existing in-vehicle networks. To address these threats, researchers have proposed two mainstream schemes, including message authentication and intrusion detection system (IDS), where the scheme of message authentication needs to occupy the limited bandwidth of controller area network (CAN) bus. Furthermore, most IDSs either cannot locate the sender of the attack, fail to detect aperiodic malicious frames, or require prior knowledge of which CAN identifiers (IDs) belong to which electronic control units (ECUs). To address these weaknesses, we propose a practical voltage-based IDS named IdentifierIDS for real in-vehicle networks. To the best of our knowledge, it is the first scheme to detect intrusions by establishing a voltage fingerprint for each ID without the need for prior knowledge. This allows IdentifierIDS to detect both periodic and aperiodic malicious frames without occupying the limited bandwidth of the CAN bus. As a self-learning IDS, it can adapt to different in-vehicle networks without the need for customization for them. Experiments on three real vehicles demonstrate the robustness of our scheme in different in-vehicle networks.

Index Terms—Intelligent connected vehicle, one-class classification, intrusion detection system, controller area network, voltage signal.

I. INTRODUCTION

AS INNOVATIVE technologies such as autonomous driving, over-the-air (OTA) technology, and vehicle-to-

everything (V2X) are widely applied to intelligent connected vehicles (ICVs), people can gain a more convenient and safer driving experience. ICVs carrying autonomous driving functions utilize sensors such as lidar, cameras, and ultrasonic radar to sense the surrounding traffic. ICVs can utilize OTA technology to download update packages via mobile communication networks (e.g., 4G and 5G) or WiFi in order to improve system functions or fix software bugs. The V2X technology allows ICVs to communicate with surrounding vehicles, people, and infrastructures, reducing traffic accidents and improving travel efficiency.

Although the application of these technologies facilitates our lives, they also bring a series of vulnerable interfaces (e.g., 5G, Bluetooth, and WiFi), which pose a significant security threat to the existing in-vehicle network. An attacker can use these interfaces to compromise an electronic control unit (ECU) such as telematics box, in-vehicle infotainment (IVI), etc. Moreover, the absence of a message authentication mechanism of in-vehicle networks, such as the controller area network (CAN) bus, results in the receiver ECU's inability to verify the legitimacy of messages. This means that an attacker can utilize a compromised ECU to send malicious frames to control more critical functions. Such concerns are real and urgent. In 2016, Miller and Valasek [1] discovered vulnerabilities in Jeep and then controlled the vehicle through remote access points of IVI, including volume adjustment, engine shutdown, and tire steering. In 2018, Tencent Keen Lab [2] exploited a bypass vulnerability to attack the gateway ECU, thereby compromising the body control ECU. In 2021, Tencent Keen Lab [3] remotely sent malicious frames through browser vulnerability in IVI, controlling the lighting and sunshade.

To deal with these threats, two mainstream schemes have been proposed. One is to use message authentication code (MAC) to verify the reliability of the frame [4], [5], [6], [7], [8], [9]. However, MAC needs to occupy storage bits, while the maximum length of the message's data field is 64 bits. So applying MAC requires sending additional frames, which increases the bus load and threatens the vehicle. Another solution, which is to design an intrusion detection system (IDS) to detect malicious frames without consuming additional CAN bus bandwidth, has attracted more and more attention from researchers.

There are three main types of IDSs: parameter monitoring-based IDS, information theory-based IDS, and fingerprint-based IDS. The parameter monitoring-based IDS mainly includes observing the traffic fluctuations of messages in the bus to detect attack [10], [11], [12], [13] or analyzing the

Manuscript received 15 January 2023; revised 12 July 2023 and 16 October 2023; accepted 17 October 2023. Date of publication 23 October 2023; date of current version 22 November 2023. This work was supported in part by the National Key Research and Development Program of China under Grant 2022YFB3104200, in part by the National Natural Science Foundation of China under Grant 62202386, in part by the China Postdoctoral Science Foundation under Grant 2023M732861, in part by the 2022 Suzhou Innovation and Entrepreneurship Leading Talents Program (Young Innovative Leading Talents) under Grant ZXL2022458, in part by the Basic Research Programs of Taicang under Grant TC2021JC31 and Grant TC2022JC22, in part by the Fundamental Research Funds for the Central Universities under Grant D5000210817, and in part by the Special Funds for Central Universities Construction of World-Class Universities (Disciplines) and Special Development Guidance under Grant 0639023GH0202091 and Grant 0639023SH0201091. An earlier version of this paper was presented in part at the 2022 IEEE Global Communications Conference [DOI: 10.1109/GLOBE-COM48099.2022.10000766]. The associate editor coordinating the review of this manuscript and approving it for publication was Prof. Haijun Zhang. (Corresponding author: Jiajia Liu.)

The authors are with the National Engineering Laboratory for Integrated Aero-Space-Ground-Ocean Big Data Application Technology and the School of Cybersecurity, Northwestern Polytechnical University, Xi'an, Shaanxi 710072, China (e-mail: liujiajia@nwpu.edu.cn).

Digital Object Identifier 10.1109/TIFS.2023.3327026

delay between regular frames and remote frames to detect intrusion [14]. The information theory-based IDS mainly detects abnormality by monitoring the fluctuation of information entropy on CAN bus [15], [16] or calculating Hamming distance [17], [18], [19]. As far as we know, neither parameter monitoring-based IDS nor information theory-based IDS can locate the senders of malicious frames. This allows the compromised ECU to attack the vehicle at any time in the future until the ECU is removed. To address this weakness, the fingerprint-based IDS is suggested. There are some researchers who use clock skew to establish fingerprints for ECUs [20], [21], [22]. However, clock-based IDS cannot detect aperiodic identifier (ID) messages, while attackers can send malicious frames with arbitrary IDs. Some researchers have proposed the voltage-based IDS [23], [24], [25], [26], [27], [28], [29], [30], [31] that can detect malicious frames with both periodic and aperiodic ID.

The comparison of the current mainstream voltage-based IDSs is shown in Table I. To the best of our knowledge, the first scheme using the characteristics of the electrical signal to establish fingerprints for ECUs was proposed by Murvay and Groza [29]. This scheme could identify the sender ECU of the signal on the CAN bus. In 2017, Cho and Shin [23] proposed Viden and validated the scheme on real vehicles. Viden can accurately locate the compromised ECU, but has limitations under electromagnetic interference. In 2018, Choi et al. [24], [25] used the differential voltage signal to establish fingerprints for ECUs, which allows IDS to have the anti-interference ability in the electromagnetic environment. Unfortunately, IdentifierIDS has limitations in detecting malicious frames with non-diagnostic IDs due to the unknown mapping between ECUs and IDs. Subsequently, Scission was proposed by Kneib and Huth [26] and can accurately detect multiple attacks. The difference to previous schemes lies in the fact that Scission can detect malicious frames from additional nodes by calculating the probability that the frame to be tested belongs to all ECUs. In 2019, Foruhandeh et al. [27] proposed SIMPLE that required only one malicious frame to detect the Hill-climbing-style attack. SIMPLE was designed to update fingerprints based on environmental changes, such as temperature and voltage variations. In 2020, Kneib et al. [28] proposed EASI, which can detect malicious frames with only low hardware cost. EASI collects voltage measurements of multiple rising edges at a low sampling rate to form a composite rising edge with rich measurement, where the actual sampling rate depends on the payload length of the frame. In 2021, Liu et al. [31] proposed a simple IDS named vProfile, which directly utilized voltage measurements instead of computed features to establish fingerprints and detect intrusions.

These IDSs require establishing voltage fingerprint for each ECU to detect intrusion, and the knowledge of mapping between ECUs and IDs is required during both the training and testing phases. Unfortunately, to protect the in-vehicle network, vehicle manufacturers do not disclose the knowledge [25]. In addition, due to the variation in mappings across different vehicle models, it is necessary to customize IDS for them.

To address these challenges, we propose IdentifierIDS, a scheme to establish a voltage fingerprint for each ID without prior knowledge of the mapping between ECUs and IDs. Our work follows the observation that messages with different IDs from a same ECU have the same voltage characteristic, while messages with IDs of different ECUs have different characteristics. During the training phase, our system can directly collect frames for each ID to establish ID fingerprints because the ID of each frame is unconcealed. In the testing phase, our system matches the test frame with the corresponding fingerprint of its ID. If the match fails, it can be identified as a malicious frame. This allows IdentifierIDS to function as a self-learning IDS that can adapt to different vehicle models without knowing the prior knowledge of the mapping. To reduce the number of sampling points, our system only collects the first rising edge, the first dominant bit, and the first falling edge after the arbitration field of each frame during testing phase. To avoid occupying the limited computational resources of ECUs, IdentifierIDS can be connected to in-vehicle network as an independent additional device to detect intrusions. A part of this work was presented in [32]. Compared to the preliminary version of this work, in this paper, we improve the scheme to detect bus-off attack and malicious frames from additional nodes. Meanwhile, we provide more technical details about the detection process. Besides, we also enrich the dataset (including more ECUs and more IDs) and evaluate our scheme on more real vehicles. We further select a new algorithm and compare its time cost with other algorithms.

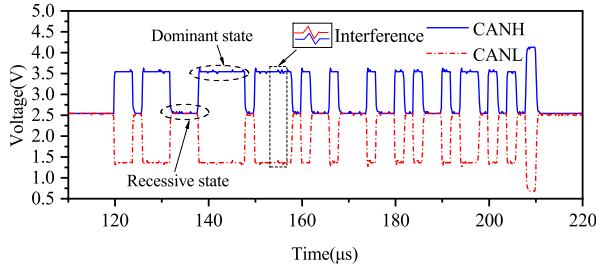
The contributions of this paper can be summarized as follows:

- We propose a practical IDS based on establishing a voltage fingerprint for each ID. It can detect intrusions without the prior knowledge of the secret mapping between ECUs and IDs.
- IdentifierIDS can detect malicious frames with periodic IDs or aperiodic IDs.
- IdentifierIDS, as a self-learning IDS, can adapt to different in-vehicle networks without the need for customization for different vehicle models.
- We evaluate the performance of IdentifierIDS in detecting multiple attacks using frames collected from three real vehicles, where the average accuracy of detecting compromised ECUs exceeds 99.1%, and the accuracy of detecting the additional node is 100%.

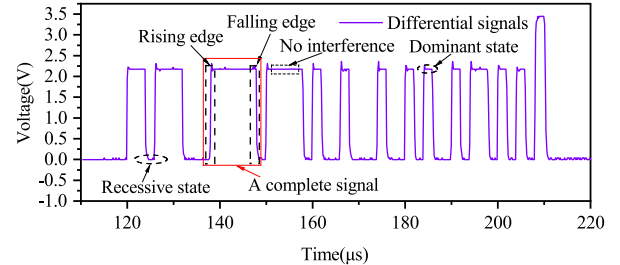
The remainder of this paper is organized as follows. We introduce the CAN bus protocol, voltage fingerprints, threat model, and Gaussian mixed model (GMM) in Section II. Next, we introduce the collection and preprocessing of voltage signal, extraction of features, an example of distinguishing the homologous frame, and detection of multiple attacks in Section III. Then, the detailed setup of the experiment is introduced in Section IV. Subsequently, we discuss the stability of the IDS, how it helps to locate the compromised device, and the device of deploying IdentifierIDS in Section V. Finally, the paper is summarized in Section VI.

TABLE I
COMPARISON OF MAINSTREAM VOLTAGE-BASED IDSs

Index	Viden CCS - [23]	Choi <i>et al.</i> TVT - [24]	VoltageIDS TIFS - [25]	Scission CCS - [26]	SIMPLE ACSAC - [27]	EASI NDSS - [28]	IdentifierIDS
Signal	high, low	differential	differential	differential	differential	differential	differential
Fingerprint	ECU	ECU	ECU	ECU	ECU	ECU	ID
Classification algorithm	-	multi-class	multi-class	multi-class	-	multi-class	one-class
Mapping between IDs and ECUs	need	need	need	need	need	need	needless
Platform	real vehicle	prototype	real vehicle	real vehicle	real vehicle	real vehicle	three real vehicles
Identify compromised ECU	99.8%	96.5%	94.1/61.0%	99.9%	99.1%	99.9%	99.4/99.4/99.1%
Identify additional node	×	100%	×	-	-	-	100%
Detect bus-off attack	×	×	100%	×	×	×	100%



(a) Voltage signals of CAN high and CAN low



(b) Differential voltage signal

Fig. 1. The voltage signal of a data frame.

II. BACKGROUND

In this section, we first introduce the basics of CAN bus, including the origin of CAN bus, differential signals, and arbitration mechanism. Next, we introduce the mapping relationship between ECUs and IDs, which is not disclosed by the vehicle manufacturer. Then, we discuss the differences between ECU fingerprints and ID fingerprints in terms of intrusion detection. Finally, we introduce multiple attack models.

A. Controller Area Network

Bosch proposed the CAN bus protocol in 1986, renowned for its exceptional performance and reliability. The CAN bus is widely used in numerous industries, including automotive, aviation, and industrial control. Since 2008, the CAN bus protocol has been mandated as a standard for American vehicles and light trucks.

1) *Differential Signal*: The CAN bus consists of two twisted pairs known as CANH and CANL. In Fig. 1(a), data frame bits are transmitted as high and low levels on the CAN bus. When sending a recessive state (logical 1), both CANH and CANL are at 2.5 V. Transmitting a dominant state (logical 0) involves raising CANH by 1 V and lowering CANL by 1 V. It is worth mentioning that the ECU transmits messages through differential signals instead of CANH and CANL. The differential signal, which is the difference between CANH and CANL, has better anti-interference ability in the electromagnetic environment [26]. As shown in Fig. 1(a), the interference on the CAN bus usually exists in common mode, and CANH and CANL will be affected in the meantime.

It means that the differential signal, as the difference between CANH and CANL, can cancel the interference. In addition, when the electrical signal switches between dominant and recessive states, the voltage differential signal generates rising and falling edges. As shown in Fig. 1(b), a complete signal composed of these two edges and the dominant state contains the characteristic of the sender ECU, and a message contains multiple complete signals.

2) *Arbitration Mechanism*: There are many ECUs connected to the CAN bus. The bus utilizes the arbitration mechanism to resolve transmission collisions when multiple ECUs want to send messages simultaneously. The mechanism stipulates that an ECU can occupy the bus only when the channel is idle. However, multiple ECUs could send messages simultaneously when bus is free, which can lead to a collision on CAN bus. Therefore, the CAN bus needs to determine the priority of each message to avoid a collision. A standard frame format is shown in Fig. 2, where ID represents the priority of a data frame. The lower the value of an ID, the higher its priority. Furthermore, since IDs that ECUs can send do not intersect, it ensures that no multiple frames with the same priority compete for the bus. When multiple ECUs send data frames together, the CAN bus compares IDs of these messages bit by bit. Once a particular bit of different messages has different values, the message whose bit is a recessive state (logical 1) loses arbitration and will be stopped.

3) *Mapping Between ECUs and IDs*: The frames sent by ECUs are broadcast to all ECUs on the CAN bus. The ID of each frame represents its unique sender ECU and the meaning of the data field. Furthermore, other ECUs decide whether to

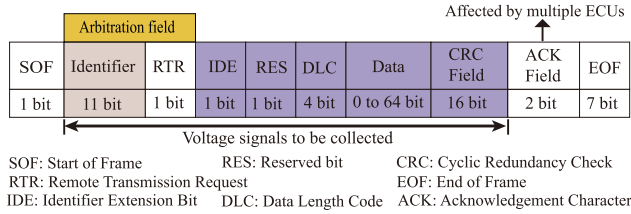


Fig. 2. A standard frame of the CAN bus.

receive the frame based on its ID. Although modern vehicles may have hundreds of ECUs and IDs, there are only a few ECUs directly connected to the CAN bus. According to the study conducted by Popa et al. [33] on 9 modern vehicles, the average number of ECUs on the CAN bus of these vehicles is 6, and the average number of unique IDs is 44.

B. ECU Fingerprints and ID Fingerprints

Variations in manufacturing environments, cable lengths, and wiring resistances among different ECUs on the CAN bus lead to distinct voltage characteristics in their data frames. Based on this, mainstream voltage-based IDSs [23], [24], [25], [26], [27], [28], [29], [30], [31] can identify the sender ECU of messages and detect intrusions by establishing unique voltage fingerprint for each ECU.

The strong capability of these IDSs in detecting intrusions relies on prior knowledge of which IDs belong to which ECUs. Specifically, during the training phase, these schemes require collecting training frames for each ECU, while each frame does not carry its source information. These schemes need to determine which ECU the frame originates from through its ID, which relies on the prior knowledge of which IDs belong to which ECUs. During the testing phase, these IDSs predict the sender ECU of the test frame. If the frame's ID does not belong to the predicted ECU, it can be considered a malicious frame, which also relies on prior knowledge. Vehicles from different manufacturers, or even different vehicle models from the same manufacturer, typically have different in-vehicle networks. As a result, these IDSs require customization for each vehicle model in order to determine which ID frames originate from the same ECU.

Vehicle manufacturers do not disclose this knowledge of mapping. To get the mapping, approaches of these works mainly include: using the clock skew of the ECU to get the mapping [20], [21] and using the diagnostic ID to map an ID for each ECU. It is worth noting that these mapping approaches cannot determine the sender ECUs of partial IDs (e.g., on-event IDs [33]), rendering these IDSs incapable of detecting the malicious frames with these IDs. To address these limitations, IdentifierIDS detects intrusions by establishing ID fingerprints, eliminating the need for this prior knowledge. Since each ECU can use non-overlapping IDs, frames with the same ID exhibit identical voltage characteristics. The training and testing phases of IdentifierIDS do not require this knowledge. More specifically, for the training phase of IdentifierIDS, establishing fingerprints for an ID can be achieved by directly collecting frames with the ID on the CAN bus because the ID

of each frame is unconcealed. In the testing phase, our system matches the test frame with the corresponding fingerprint of its ID. If the match fails, it can be identified as a malicious frame. Since this matching uses the fingerprint to score the test frame, it does not rely on the prior knowledge of mapping. This allows IdentifierIDS to function as a self-learning IDS that can adapt to different in-vehicle networks. When a sufficient number of signal records are available for an ID, our system can directly establish a fingerprint for that ID.

C. Threat Model

We consider four attacks that seriously threaten in-vehicle network security: fabrication attack, masquerade attack, additional attack, and bus-off attack.

1) *Fabrication Attack*: The first attack type we consider is to compromise an in-vehicle ECU to send a large number of malicious frames with a specific ID and malicious data field, which was defined by Cho and Shin [20] as the fabrication attack. After compromising a target ECU, the attacker controls it to send malicious frames of a specific ID with high frequency. These malicious frames are so numerous that they can overwrite commands from the normal sender ECU. At this point, the attacker can control the functions of almost all ECUs of the CAN bus.

2) *Masquerade Attack*: Masquerade attack, defined by Cho and Shin [20], can evade most frequency-based IDS. The purpose of the masquerade attack is to mask the damaged ECU and manipulate its functions. First, the attacker needs to compromise an ECU, thereby listening to the CAN bus and recording the periods of the target ECU's ID messages. Then, the attacker damages the target ECU without being able to control the ECU. Finally, the compromised ECU sends the message with the previously recorded period. This allows malicious frames to bypass most frequency-based, because masquerade attack does not change the normal period of CAN bus messages.

3) *Additional Attack*: Next, we focus on the attack that connects an additional node to the CAN bus through the OBD-II port, and the node can send malicious frames with any ID. Unlike the fabrication attack and masquerade attack, the sender of the malicious frame does not originate from the vehicle's ECU but from an unknown node. This means that physical characteristics carried by malicious frames are different from ECU fingerprints stored by voltage-based IDS and clock-based IDS.

4) *Bus-off Attack*: Finally, we consider a powerful attack referred to as the bus-off attack [34], which has the capability to force the ECU offline. This leverages the bus-off mechanism of the CAN bus. Specifically, when an ECU encounters an error while transmitting a message, its transmit error counter (TEC) is incremented by 8. Similarly, when an ECU encounters an error while receiving a message, its receive error counter (REC) is incremented by 8. Once the TEC of an ECU exceeds 255, it enters the bus-off state, causing it to cease both message reception and transmission.

Fig. 3 shows an example of the bus-off attack. The attacker intends to induce bit error in the victim ECU to increase its TEC. When an ECU sends a recessive bit after the arbitration

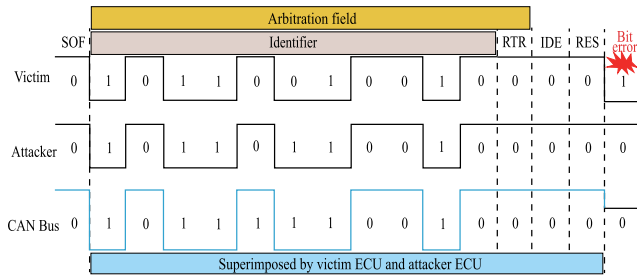


Fig. 3. An example of the bus-off attack.

field but detects a dominant bit on CAN bus, the ECU generates a bit error. Considering that the RTR bit, identifier extension (IDE) bit, and reserved (RES) bit of a normal data frame are dominant, the appropriate place to force the victim ECU to encounter a bit error is after the RES bit. It can be determined that the voltage signal of IDE bit and RES bit is superimposed by the attacker ECU and the victim ECU. Through multiple rounds of attacks, attacker ECU can force the victim ECU into the bus-off state.

D. GMM Model

In recent years, machine learning has developed rapidly and is widely used in data mining, pattern recognition, and natural language processing. In addition, many researchers are considering using machine learning in the field of information security [35] (e.g., intrusion detection, Trojan detection, and vulnerability scanning).

As far as we know, most IDSs based on establishing voltage fingerprints for ECUs use multi-class classification algorithms such as support vector machine [24], [25] and logistic regression [26], [28] to establish a model for protecting in-vehicle networks. As mentioned in Subsection II-B, it is challenging to obtain all frames of each ECU to establish voltage fingerprints because the mapping relationship between ECUs and IDs is unknown. To overcome this weakness, the GMM is used in this paper to establish a voltage fingerprint for each ID. GMM is a weighted combination of multiple Gaussian distribution models and can be used for clustering and detecting outliers. During model iteration, GMM can fit the concentrated and continuous data within the training dataset to enhance the model’s ability to generalize the distribution characteristics of the training data effectively. We use GMM to train all samples of each ID to form the Gaussian cluster and learn their parameters. When examining a message, the model outputs the local Mahalanobis distance of the message to the centroid of the cluster. The larger the distance, the more likely the message is an outlier. If a model outputs a score to a frame higher than predefined threshold of the model, the frame’s sender does not match the ID associated with that model.

III. OUR METHOD

In this section, we introduce three components of IdentifierIDS: voltage signal collection and preprocessing, feature extraction, and intrusion detection.

A. Voltage Signal Collection and Preprocessing

First, IdentifierIDS collects voltage signals from frames on the CAN bus. As shown in Fig. 1, the common mode interference from CANH and CANL is canceled out on the differential signal. The differential signal is the difference between CANH and CANL. This means that the hardware resources required for storing the measurement points of 1-bit CANH and CANL signals are the same as those required for 2-bit measurement points of the differential signal. Therefore, lower hardware resources are required to store the differential voltage value. IdentifierIDS records measurements of each differential voltage signal at a sampling rate of 62.5 MS/s.

In order to establish accurate voltage fingerprint, it is necessary to ensure that the signal being collected is only influenced by its sender. However, partial voltage signals of the ID field are superimposed by multiple ECUs during the arbitration. In addition, the voltage of acknowledge (ACK) is also affected by multiple ECUs. Rising edge and falling edge will occur on differential signals when the transmitted signal voltage switches between a dominant state (0) and a recessive state (1). These two edges and a dominant state represent a complete signal, as described in Section II. It is observed that signals from different positions within the same frame exhibit only marginal differences in their characteristics [28]. IdentifierIDS collected the first rising edge, the first dominant bit, and the first falling edge after the arbitration field during the testing phase, because the collected signal is sufficient to reflect the voltage characteristics of the frame. This also implies that the system does not need to collect other rising edges, falling edges, and dominant bits at other positions within the frame, thus reducing the number of sampling points and the time overhead of signal collection. Besides, our system collected all signals from the ID field to the CRC field during the training phase. The ID field was decoded to label the collected sample, and signals from the remote transmission request (RTR) field to the cyclic redundancy check (CRC) field were collected to extract voltage features. This enables our system to establish fingerprints quickly. Each collected signal is treated as an independent sample, allowing for the collection of multiple samples from a single frame, thereby reducing the number of frames required for training data collection.

The segment from the first measurement point of the complete signal to the measurement point of stable voltage is considered as the rising edge. The falling edge is a decreasing sequence containing the last point of the complete signal. The remainder of the voltage sample set of a complete signal is the dominant state.

B. Feature Extraction

Extracting features from a long sequence of voltage can potentially hide the characteristics of the signal’s sender [26]. Therefore, IdentifierIDS extracts features from the rising edge, falling edge, and dominant state, respectively. To extract these voltage features, we focus on statistical time domain features and consider statistical frequency domain features. Statistical time domain features can describe signal strength variation over time, while statistical frequency domain features can

TABLE II

STATISTICAL TIME AND FREQUENCY DOMAIN FEATURES. $s(i)$ IS THE SPECTRUM OF THE SIGNAL. $f(i)$ IS THE FREQUENCY OF THE SIGNAL SPECTRUM. $t(i)$ IS THE VOLTAGE VALUE OF SIGNAL

Feature	Description
Statistical time domain feature	
Max	$t_{max} = \text{Max}\{t(i) _{i=1 \dots N}\}$
Min	$t_{min} = \text{Min}\{t(i) _{i=1 \dots N}\}$
Mean	$\mu_t = \frac{1}{N} \sum_{i=1}^N t(i)$
Standard deviation	$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (t(i) - \mu_t)^2}$
Mean deviation	$MD = \frac{1}{N} \sum_{i=1}^N t(i) - \mu_t $
Root mean square	$RMS = \sqrt{\frac{1}{N} \sum_{i=1}^N (t(i))^2}$
Skewness	$skewness_t = \frac{1}{N} \sum_{i=1}^N \left(\frac{t(i) - \mu_t}{\sigma} \right)^3$
Kurtosis	$kurtosis_t = \frac{1}{N} \sum_{i=1}^N \left(\frac{t(i) - \mu_t}{\sigma} \right)^4$
Statistical frequency domain feature	
Centroid	$\mu_f = \frac{\sum_{i=1}^F f(i)s(i)}{\sum_{i=1}^F s(i)}$
Entropy	$E = \sum_{i=1}^F w_i \cdot \log_2 w_i, \text{ where } w_i = \frac{s(i)}{\sum_{i=1}^F s(i)}$
Spread	$spread = \sqrt{\sum_{i=1}^F ((f(i) - \mu_f)^2 \cdot \omega_i)}$
Skewness	$skewness_f = \frac{\sqrt{\sum_{i=1}^F (f(i) - \mu_f)^3 \cdot \omega_i}}{(\sum_{i=1}^F ((f(i) - \mu_f)^2 \cdot \omega_i))^{\frac{3}{2}}}$
Average	$average = \frac{\sum_{i=1}^F s(i)}{F}$
Kurtosis	$kurtosis_f = \frac{\sqrt{\sum_{i=1}^F (f(i) - \mu_f)^4 \cdot \omega_i}}{(\sum_{i=1}^F ((f(i) - \mu_f)^2 \cdot \omega_i))^2}$
Irregularity	$irregularity = \frac{\sum_{i=1}^{F-1} (s^2(i) - s^2(i+1))^2}{\sum_{i=1}^{F-1} (s^2(i))^2}$
Variance	$variance = \frac{(\sum_{i=1}^F s(i) - average)^2}{F-1}$

show which single-frequency signals make up the signal. In the past few decades, there have been many studies on signal features [25], [36], [37], [38]. These experimental works are significant for establishing voltage fingerprints.

As shown in Table II, VIII statistical time domain features and 8 statistical frequency domain features are considered to identify the sender of a frame. Statistical time domain features can be obtained directly from measurements. Max (t_{max}) and Min (t_{min}) are the highest and lowest points of a voltage measurement sequence. Mean (μ_t) represents the voltage mean of the N measurements ($t(i)$). Standard deviation (σ) describes the degree of dispersion of the acquired voltage signal. Mean deviation (MD) is used to determine the degree to which the measured value is off-target from its mean. Root mean square (RMS) is also called the effective value, representing the ECU's ability to send the signal power. Skewness ($skewness_t$) could measure the degree of asymmetry in the distribution of measured values. Kurtosis ($kurtosis_t$) reflects the sharpness of the statistical time domain voltage peak. To obtain statistical frequency domain features, IdentifierIDS converts statistical time domain signals into statistical frequency domain signals using Fourier transform. Centroid (μ_f) is used to describe the center of gravity of the signal spectrum. Entropy (E) represents the energy concentration of the spectrum. Spread ($spread$) is described as the dispersion of the frequency components of the measurements around the centroid (μ_f). Skewness ($skewness_f$) represents the spectrum's direction and degree of tilt. Average ($average$) is the mean of the signal

TABLE III

SELECTED FEATURES FOR ONE-CLASS CLASSIFICATION

Features	Rising edge	Dominant state	Falling edge	Features	Rising edge	Dominant state	Falling edge
t_{max}	-	✓	-	μ_f	✓	-	✓
t_{min}	-	✓	✓	E	-	-	✓
μ_t	✓	✓	-	$spread$	-	✓	✓
σ	✓	-	✓	$skewness_f$	-	✓	✓
MD	✓	✓	✓	$average$	-	✓	✓
RMS	-	-	✓	$kurtosis_f$	✓	-	✓
$skewness_t$	✓	-	✓	$irregularity$	-	✓	-
$kurtosis_t$	✓	✓	✓	$variance$	-	✓	-

spectrum. Kurtosis ($kurtosis_f$) represents the flatness of the spectral distribution. The degree of variation of the spectral continuum peaks can be demonstrated in terms of irregularity ($irregularity$). Variance ($variance$) represents the degree of dispersion of the statistical frequency domain data.

Feature selection is an essential step for machine learning models. Selecting appropriate features can reduce the computational overhead and improve recognition accuracy. Therefore, we use the feature filter proposed in [39] to select features for one-class classification algorithm. This scheme ranks the importance of features on 5 indicators (including spectral score, information score, Pearson correlation, intra-class distance, and interquartile range), which can be used to select features of great significance to the model. As shown in Table III, we selected 29 features, where the number of features extracted from the rising edge, dominant state, and falling edge was 7, 10, and 12, respectively.

C. Example of Distinguishing Homologous Frame

Fig. 4 illustrates the scores outputted when matching ID fingerprint (model) with frames from other ECUs. It is worth noting that IdentifierIDS does not need to know the mapping relationship between ECUs and IDs of in-vehicle networks. To demonstrate that the fingerprint established by IdentifierIDS can identify the sender of message, the approach in [21] was used to know that {0x329, 0x316} came from the same ECU, {0x39A, 0x39E} were from the same ECU, {0x340, 0x34A} came from the same ECU, and {0x360, 0x350} also came from the same ECU.

Fig. 4 shows the distribution of scores for data frames on the fingerprints corresponding to IDs 0x316, 0x340, and 0x350. In Fig. 4(a), we observed that only frames with IDs 0x316 and 0x329 scored below 220 on the fingerprint corresponding to 0x316. This verifies that different IDs from the same ECU have the same voltage fingerprint. Fig. 4(b) and Fig. 4(c) also show that the scores of ID frames from other ECUs hardly ever intersect with the scores of frames from the sender of the model's ID. Therefore, a model trained on an ID can detect whether a frame is homologous to the ID.

D. Fabrication Attack and Masquerade Attack Detection

For IdentifierIDS, there is no difference between detecting fabrication attack and masquerade attack. As an unsupervised

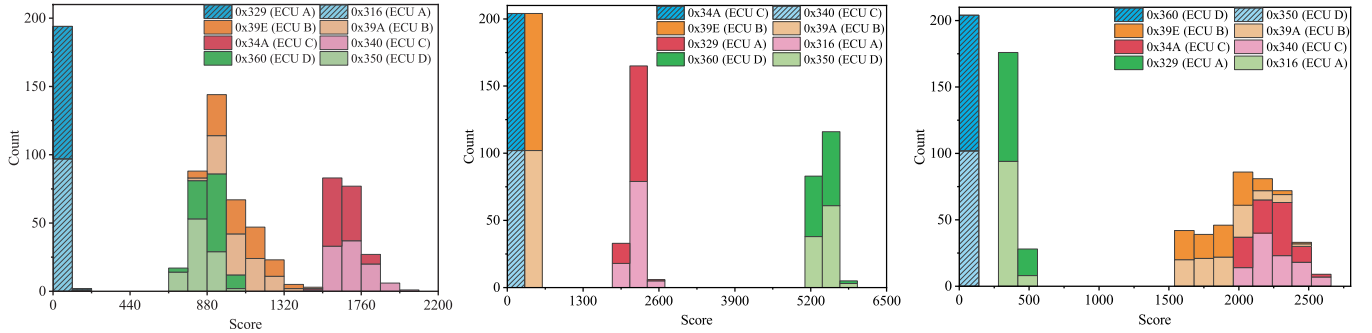


Fig. 4. The score distribution of frames with different IDs on models for 0x316, 0x340, and 0x350 on Luxgen.

learning algorithm, GMM can be used to detect anomalies. During the training phase, IdentifierIDS employs GMM to train a model for each ID. This model is capable of discerning whether a frame originates from the same sender as the ID associated with the model. In the testing phase, IdentifierIDS puts the frame to be tested into the frame's ID model to detect attacks. When an attack is detected, the malicious frame is matched against other models to identify compromised ECU.

The detailed steps of training fingerprints and detecting intrusions are shown in Algorithm 1. During the model training phase, IdentifierIDS first decodes the ID of each training sample and extracts the voltage features ($[f_1, f_2, f_3 \dots f_{29}]$) from the rising edge, dominant state, and falling edge respectively (lines 3 to 10). Then, the system proceeds to establish ID fingerprints. Samples with the same ID (id_x) are grouped into the individual dataset ($Training\ set(x)$), with the number of datasets being equal to the number of IDs (lines 12 and 13). IdentifierIDS utilizes the GMM algorithm to individually train each dataset and obtain its respective ID fingerprint (line 14). Our system determines the order of training ID fingerprints based on the speed at which ID finishes collecting training data. IDs with shorter cycles are typically prioritized for fingerprint establishment. In the detection phase, 29 features and the ID of an electrical signal are collected (lines 18 to 21). The *Model* corresponding to the *id* is preferentially selected to score this signal (line 22). The frame is normal if the *score* is below the set threshold (lines 23 and 24). Otherwise, the message is malicious. In order to locate the sender of the malicious message, other models in the model set would, in turn, score the message until the $score_i$ of $Model_i$ is lower than the threshold. At this point, the malicious message has the same sender as the id_i corresponding to $Model_i$ (line 30). It is worth mentioning that once a normal frame successfully matches its corresponding ID fingerprint, there is no need to match it with other ID fingerprints. This means that most frames on the CAN bus will only be matched with one ID fingerprint.

Fig. 5 shows an example of matching test frame to ID fingerprints. Fingerprint 1 and fingerprint 4 originate from the same ECU, while fingerprint 2 and fingerprint 3 belong to other two different ECUs. The frame with ID 2 is first matched against fingerprint 2, and if the match fails, it is considered a malicious frame. Based on this, IdentifierIDS could detect malicious frames without knowing the mapping between ECUs

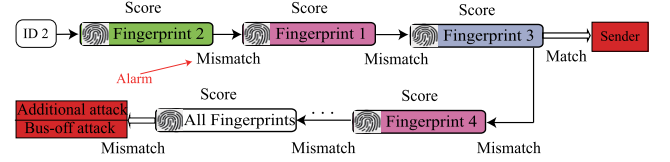


Fig. 5. An example of matching test frame to ID fingerprints.

and IDs. Next, the malicious frame is matched against other fingerprints in turn. When the malicious frame is successfully matched with fingerprint 3, the ECU corresponding to ID 3 is considered compromised. This implies that IdentifierIDS can locate the sender without prior knowledge of the mapping.

E. Additional Attack Detection

It is difficult for multi-class classification models to detect messages from unknown additional nodes. However, GMM, which is an unsupervised algorithm, can be used to detect outliers. Frames from the additional node, whose voltage characteristics are not included in the model's training set, are outliers for all models. During the testing phase, the frame to be tested will be sequentially matched with all ID models (fingerprints). Malicious frames from additional nodes are not expected to match any of the recorded fingerprints.

The detailed steps for model training and detection of additional attacks are shown in Algorithm 1. Notably, detecting additional attack does not require training the new model. When an abnormality is detected, IdentifierIDS will traverse $Model_i$ in the model set in turn to score the features of the malicious frame (lines 26 to 28). It is evident that each time the score is higher than $threshold_i$, which means that the message cannot be matched to any ECU of in-vehicle network. At this point, it can be considered that malicious frames from additional nodes are detected.

As shown in Fig. 5, a malicious frame with ID 2 from an additional node outputs a score that exceeds the threshold when matched with fingerprint 2, thus triggering an alarm. Subsequently, the frame is matched against other fingerprints in turn. Because IdentifierIDS does not employ voltage signals from additional nodes for fingerprint establishment, the malicious frame will indeed fail to match with any of the fingerprints.



Fig. 6. Frames were collected from three real vehicles.

F. Bus-off Attack Detection

As shown in Subsection II-C, the voltage signal of IDE bit and RES bit is superimposed by the attacker ECU and the victim ECU. In the phase of detecting intrusion, IdentifierIDS extracts the first rising edge, the first dominant bit, and the first falling edge after the arbitration field, where the first dominant bit is IDE bit. Considering that the attacker may induce bit error at arbitrary position after the RES bit, we cannot guarantee that the collected rising edge and falling edge are corrupted (superimposed). Therefore, we consider the worst-case scenario where the IdentifierIDS collects a signal where only the dominant bit is corrupted, and the two edges are normal.

Similar to detecting additional attack, detecting bus-off attack does not require training new ID fingerprints. The frames from the bus-off attack will not match any ID fingerprint due to the significant difference between the collected corrupted dominant bit and the normal signal.

IV. EXPERIMENTAL RESULTS

In this section, we evaluate the performance of IdentifierIDS in detecting multiple attacks using frames collected from real vehicles.

A. Experimental Environment

1) *Actual Vehicles and Equipment*: To demonstrate that IdentifierIDS is suitable for real in-vehicle networks, we evaluated its performance on frames collected from three real vehicles: a Buick in Fig. 6(a), a Luxgen in Fig. 6(b), and a Toyota in Fig. 6(c). To record the frames on these vehicles, we connected the PicoScope 2206B to the CAN bus through the OBD-II port and collected electrical signals at a sampling rate of 62.5 MS/s. Furthermore, we used the CANalyst-II as an additional node to generate malicious frames.

The PicoScope 2206B, with a maximum sampling rate of 500 MS/s, is indeed more expensive laboratory equipment. It should be noted that there are affordable sampling devices available that meet the requirements of IdentifierIDS, such as the ADC12DL080 [40] from Texas Instruments. It supports a maximum sampling rate of 80 MS/s and is priced below \$20. Therefore, the device suitable for deploying IdentifierIDS should not be excessively expensive, which implies that deploying IdentifierIDS on external device to protect the in-vehicle network is both available and meaningful.

TABLE IV

THE IDS SELECTED TO EVALUATE IDENTIFIERIDS ON THE THREE VEHICLES

Vehicle	ID
Buick	0x0AA, 0x0BE, 0x0C9, 0x0F9, 0x0B9, 0x0BA, 0x0BB, 0x0BC, 0x0C1, 0x0C5, 0x0D1, 0x1C6, 0x0F1, 0x1E1
Luxgen	0x316, 0x329, 0x335, 0x33A, 0x340, 0x34A, 0x34F, 0x350, 0x360, 0x380, 0x38A, 0x39A, 0x39E
Toyota	0x020, 0x0B4, 0x0BA, 0x1AA, 0x320, 0x1C8, 0x3F9, 0x1D0, 0x260, 0x262, 0x387, 0x38E, 0x38F, 0x443

2) *Hardware and Software Environment*: We decoded the acquired data frames using PicoScope 6 software to obtain the ID of each frame. To estimate the performance of IdentifierIDS on resource-constrained devices, we deployed our system on a Raspberry Pi 4 Model B [41], which was released in 2019 and priced at \$45. It has a Broadcom BCM2711, Quad-core CortexA72 (ARM v8) 1.5GHz processor with 2 GB SDRAM. All code was written based on Python 3.7. We used multiple one-class classification algorithms in the Pyod library [42] to train the IdentifierIDS model. We utilized the Python code [43] opened in scheme [21] to determine which IDs come from which ECUs.

3) *Datasets*: We recorded 500 frames for each ID both from the vehicle after cold-starting and after 2 hours of runtime. One week later, we replicated the data collection procedure as previously conducted. After data preprocessing, IdentifierIDS retained 15000 signals from 2000 frames for each ID. During the training phase, 14000 signals for a certain ID were used as the training set to train the model for that ID. It is worth noting that the experimental results in Subsection IV-C demonstrate that accurate intrusion detection can be achieved by establishing voltage fingerprints with just 6900 signals. In the testing phase, the remaining 1000 signals of this ID and all signals of other IDs would be used as the test set. Finally, we collected 10000 signals for each vehicle from additional node as the test set for detecting the additional attack.

B. Fingerprinting IDs

It is worth noting that IdentifierIDS does not require the mapping between ECUs and IDs to detect multiple attacks. However, to evaluate the performance of the ID fingerprints in identifying the senders of frames, we used the approach proposed in [21] to map the sender ECU for the IDs in

Algorithm 1 Intrusion Detection

```

1: function MODEL TRAINING(T: a set of training frames)
2:   for  $i = 1$  to  $|T|$  do
3:      $id_i \leftarrow \text{Decode}(t_i) \text{ /* } t_i \in T \text{ */}$ 
4:      $\text{ID set}(i) \leftarrow id_i$ 
5:      $(R_i, D_i, F_i) \leftarrow \text{Preprocess}(t_i)$ 
6:      $\text{/* } R_i : \text{rising edge, } D_i : \text{dominant state, } F_i : \text{falling edge */}$ 
7:      $[f1, f2, f3 \dots f7] \leftarrow \text{Feature Extraction}(R_i)$ 
8:      $[f8, f9, f10 \dots f17] \leftarrow \text{Feature Extraction}(D_i)$ 
9:      $[f18, f19, f20 \dots f29] \leftarrow \text{Feature Extraction}(F_i)$ 
10:    Training samples  $\text{set}(i) \leftarrow [f1, f2, f3 \dots f29, id_i]$ 
11:    Single ID set  $\leftarrow \text{Deduplicate}(\text{ID set})$ 
12:    for  $id_x$  to Single ID set do
13:      Training set(x)  $\leftarrow$  The samples whose ID is  $id_x$  in the training sample set.
14:      Model set( $id_x$ )  $\leftarrow \text{Train}(\text{Training set}(x))$ 
15:    return Model set
16:
17: function ATTACK DETECTION(s: a frame to be tested)
18:    $id \leftarrow \text{Decode}(s)$ 
19:    $[f1, f2, f3 \dots f7] \leftarrow \text{Feature Extraction}(R)$ 
20:    $[f8, f9, f10 \dots f17] \leftarrow \text{Feature Extraction}(D)$ 
21:    $[f18, f19, f20 \dots f29] \leftarrow \text{Feature Extraction}(F)$ 
22:    $\text{Score} \leftarrow \text{Test}(\text{Model set}(id), [f1, f2, f3 \dots f29])$ 
23:   if  $\text{Score} < \text{threshold}$  then
24:     return Normal
25:   else
26:     for  $\text{Model}_i$  to Model set do
27:        $id_i \leftarrow$  The ID corresponding to the  $\text{Model}_i$ 
28:        $\text{Score}_i \leftarrow \text{Test}(\text{Model}_i, [f1, f2, f3 \dots f29])$ 
29:       if  $\text{Score}_i < \text{threshold}_i$  then
30:         return Abnormal,  $id_i$ 
31:   return Abnormal

```

TABLE V
MATCHING MATRIX OF DATA FRAMES OF 14 IDS ON
14 FINGERPRINTS OF BUICK

		ECU A				ECU B				ECU C				ECU D	
		0AA	0BE	0C9	0F9	0B9	0BA	0BB	0BC	0C1	0C5	0D1	1C6	0F1	1E1
ECU A	F1 (0AA)	99.5	98.6	98.1	98.7	0	0	0	0	0	0	0	0	0	0
	F2 (0BE)	98.5	99.0	98.1	98.7	0	0	0	0	0	0	0	0	0	0
	F3 (0C9)	99.2	99.3	99.5	99.4	0	0	0	0	0	0	0	0	0	0
	F4 (0F9)	98.4	98.6	98.4	98.0	0	0	0	0	0	0	0	0	0	0
ECU B	F5 (0B9)	0	0	0	0	99.5	99.6	98.9	99.5	0	0	0	0	0	0
	F6 (0BA)	0	0	0	0	98.3	98.5	98.6	98.5	0	0	0	0	0	0
	F7 (0BB)	0	0	0	0	98.8	99.6	98.0	99.5	0	0	0	0	0	0
	F8 (0BC)	0	0	0	0	98.1	99.4	98.3	99.0	0	0	0	0	0	0
ECU C	F9 (0C1)	0	0	0	0	0	0	0	0	99.5	98.5	98.2	99.2	0	0
	F10 (0C5)	0	0	0	0	0	0	0	0	99.5	99.5	98.1	99.5	0	0
	F11 (0D1)	0	0	0	0	0	0	0	0	99.6	99.0	98.5	99.7	0	0
	F12 (1C6)	0	0	0	0	0	0	0	0	99.3	98.6	98.3	99.5	0	0
ECU D	F13 (0F1)	0	0	0	0	0	0	0	0	0	0	0	0	99.0	98.9
	F14 (1E1)	0	0	0	0	0	0	0	0	0	0	0	0	98.9	99.0

Table IV. This approach determines whether the two IDs are from the same ECU by tracking the clock skew of their messages.

TABLE VI
MATCHING MATRIX OF DATA FRAMES OF 13 IDS ON
13 FINGERPRINTS OF LUXGEN

		ECU E				ECU F				ECU G				ECU H	
		316	329	335	33A	340	34A	34F	350	360	380	38A	39A	39E	
ECU E	F15 (316)	99.5	98.0	98.5	98.9	0	0	0	0	0	0	0	0	0	
	F16 (329)	99.6	99.5	98.3	99.4	0	0	0	0	0	0	0	0	0	
	F17 (335)	99.6	98.8	98.5	99.5	0	0	0	0	0	0	0	0	0	
	F18 (33A)	99.0	98.0	98.6	99.0	0	0	0	0	0	0	0	0	0	
ECU F	F19 (340)	0	0	0	0	98.0	99.6	99.2	0	0	0	0	0	0	
	F20 (34A)	0	0	0	0	98.0	99.5	98.3	0	0	0	0	0	0	
	F21 (34F)	0	0	0	0	99.3	99.6	99.0	0	0	0	0	0	0	
	F22 (350)	0	0	0	0	0	0	0	99.0	99.5	99.3	99.5	0	0	
ECU G	F23 (360)	0	0	0	0	0	0	0	99.0	98.5	99.0	99.4	0	0	
	F24 (380)	0	0	0	0	0	0	0	98.7	98.9	99.0	99.0	0	0	
	F25 (38A)	0	0	0	0	0	0	0	98.4	98.8	98.5	98.0	0	0	
	F26 (39A)	0	0	0	0	0	0	0	0	0	0	0	98.5	98.9	
ECU H	F27 (39E)	0	0	0	0	0	0	0	0	0	0	0	98.6	98.5	

TABLE VII
MATCHING MATRIX OF DATA FRAMES OF 14 IDS ON
14 FINGERPRINTS OF TOYOTA

		ECU I					ECU J			ECU K		ECU L			
		020	0B4	0BA	1AA	320	1C8	3F9	1D0	260	262	387	38E	38F	443
ECU I	F28 (020)	98.5	98.5	98.8	98.5	98.4	0	0	0	0	0	0	0	0	0
	F29 (0B4)	98.1	99.0	98.0	98.1	98.0	0	0	0	0	0	0	0	0	0
	F30 (0BA)	98.2	98.2	99.0	98.3	98.3	0	0	0	0	0	0	0	0	0
	F31 (1AA)	98.5	98.8	98.0	98.5	98.1	0	0	0	0	0	0	0	0	0
ECU J	F32 (320)	98.3	98.8	98.5	98.9	99.0	0	0	0	0	0	0	0	0	0
	F33 (1C8)	0	0	0	0	0	99.5	99.1	99.1	0	0	5.3	5.0	4.2	4.5
	F34 (3F9)	0	0	0	0	0	98.9	98.0	98.0	0	0	5.2	4.1	5.0	4.3
	F35 (1D0)	0	0	0	0	0	99.2	98.9	99.5	0	0	6.7	6.1	5.6	5.8
ECU K	F36 (260)	0	0	0	0	0	0	0	0	100	99.2	0	0	0	0
	F37 (262)	0	0	0	0	0	0	0	0	98.6	98.5	0	0	0	0
	F38 (387)	0	0	0	0	0	2.8	3.0	3.1	0	0	98.1	98.3	98.8	98.0
	F39 (38E)	0	0	0	0	0	2.3	2.3	2.1	0	0	98.4	98.5	98.6	98.7
ECU L	F40 (38F)	0	0	0	0	0	6.0	5.7	5.7	0	0	98.2	98.7	98.0	98.3
	F41 (443)	0	0	0	0	0	2.4	3.1	2.6	0	0	98.1	98.7	98.3	99.5

1) *Buick*: We first established ID fingerprints by collecting frames from the Buick. On this vehicle, we can confirm that {0x0AA, 0x0BE, 0x0C9, 0x0F9} originated from a same ECU, {0x0B9, 0x0BA, 0x0BB, 0x0BC} came from the same ECU, {0x0C1, 0x0C5, 0x0D1, 0x1C6} came from a same ECU, and {0x0F1, 0x1E1} originated from a same ECU.

Table V shows the matching matrix of frames of 14 IDs on 14 fingerprints in turn. The F1 to F14 represent fingerprints, and the columns represent the matching rate of frames on these fingerprints. The matching matrix shows the match rate of frames with different IDs when matched with all ID fingerprints in turn. The match rate represents the percentage of frames that successfully match with fingerprints. For example, the element in the x-th column and the y-th row of the matching matrix represents the proportion of successful matches of all frames with ID x to the fingerprint corresponding to ID y. It can be seen that the frames of IDs 0x0AA, 0x0BE, 0x0C9, and 0x0F9 had high match rates in the four fingerprints from F1 to F4. Because frames of these 4 IDs all originate from

TABLE VIII
MATCHING MATRIX OF FRAMES FROM THREE REAL VEHICLES ON ID FINGERPRINTS

		ECU A			ECU B			ECU C			ECU D			ECU E			ECU F			ECU G			ECU H		ECU I			ECU J			ECU K			ECU L		
		0AA	0BE	0C9	0B9	0BA	0BB	0C1	0C5	0D1	0F1	1E1	316	329	335	340	34A	34F	350	360	380	39A	39E	020	0B4	0BA	1C8	3F9	1D0	260	262	387	38E	38F		
ECU A	F1 (0AA)	100	98.2	98.7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
	F2 (0BE)	98.0	99.0	98.6	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
	F3 (0C9)	98.8	99.0	100	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
ECU B	F5 (0B9)	0	0	0	98.0	99.2	98.4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
	F6 (0BA)	0	0	0	98.9	98.0	98.6	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
	F7 (0BB)	0	0	0	98.2	99.3	98.0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
ECU C	F9 (0C1)	0	0	0	0	0	0	98.0	98.8	98.4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
	F10 (0C5)	0	0	0	0	0	0	98.8	100	98.6	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
	F11 (0D1)	0	0	0	0	0	0	99.1	98.5	98.0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
ECU D	F13 (0F1)	0	0	0	0	0	0	0	0	98.0	98.5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
	F14 (1E1)	0	0	0	0	0	0	0	0	98.3	100	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
	F15 (316)	0	0	0	0	0	0	0	0	0	0	98.5	98.5	98.9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
ECU E	F16 (329)	0	0	0	0	0	0	0	0	0	0	99.1	98.5	98.8	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
	F17 (335)	0	0	0	0	0	0	0	0	0	0	99.0	98.3	98.0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
	F19 (340)	0	0	0	0	0	0	0	0	0	0	0	0	0	98.5	98.9	98.4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
ECU F	F20 (34A)	0	0	0	0	0	0	0	0	0	0	0	0	0	98.3	99.0	98.5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
	F21 (34F)	0	0	0	0	0	0	0	0	0	0	0	0	0	98.8	99.2	98.5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
	F22 (350)	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	98.5	98.5	98.2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
ECU G	F23 (360)	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	98.5	98.0	98.5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
	F24 (380)	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	98.1	98.3	98.0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
	F26 (39A)	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	98.5	98.6	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
ECU H	F27 (39E)	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	98.2	98.5	0	0	0	0	0	0	0	0	0	0	0	0	0	
	F28 (020)	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	99.0	98.5	98.7	0	0	0	0	0	0	0	0	0	0		
	F29 (0B4)	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	98.0	99.0	98.4	0	0	0	0	0	0	0	0	0	0		
ECU I	F30 (0BA)	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	98.5	98.1	99.0	0	0	0	0	0	0	0	0	0	0		
	F33 (1C8)	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	99.5	99.0	99.0	0	0	5.1	5.8	6.3				
	F34 (3F9)	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	98.7	98.5	98.7	0	0	5.2	4.4	5.2			
ECU J	F35 (1D0)	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	99.0	99.0	99.5	0	0	6.6	5.8	5.7			
	F36 (260)	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	100	99.2	0	0	0			
	F37 (262)	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	98.5	0	0	0	0	0	0	0	0	0	0	0	98.8	98.5	0	0	0			
ECU K	F38 (387)	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	3.4	3.1	3.6	0	0	98.5	98.1	87.7		
	F39 (38E)	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1.8	1.8	1.8	0	0	98.7	98.0	87.7		
	F40 (38F)	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	3.8	3.3	3.8	0	0	98.1	98.3	99.0		

the same ECU, they have the same voltage characteristics. This means that F1, F2, F3, and F4 trained by frames of these 4 IDs are also equivalent. For Buick, we achieved an average accuracy of over 99% using IdentifierIDS and a false positive rate of 0 at this time.

2) *Luxgen*: We then used frames from Luxgen to evaluate IdentifierIDS. For Luxgen, {0x316, 0x329, 0x335, 0x33A} came from the same ECU, {0x340, 0x34A, 0x34F} came from another ECU, {0x350, 0x360, 0x380, 0x38A} came from the same ECU, and {0x39A, 0x39E} came from the same ECU. Table VI demonstrates the matching rates of frames of these IDs on 13 fingerprints in turn, where F15 to F27 represent fingerprints on Luxgen. Since 0x316, 0x329, 0x335, and 0x33A came from the same ECU, the frames of these three IDs had high matching rates in the four fingerprints from F15 to F18. The identification accuracy of IdentifierIDS on Luxgen was also over 99% with no false positives.

3) *Toyota*: The frames of Toyota were used to evaluate IdentifierIDS. For Toyota, {0x020, 0x0B4, 0x0BA, 0x1AA, 0x320} had the same source ECU, {0x1C8, 0x3F9, 0x1D0} were from another ECU, {0x260, 0x262} had the same source ECU, and {0x387, 0x38E, 0x38F, 0x443} also originated from the same ECU. The matching rates of frames of the 14 IDs on 14 fingerprints (including F28 to F41) are shown in Table VII.

IdentifierIDS achieved an average accuracy of over 99% and an average false positive rate of less than 0.6% on Toyota.

4) *Buick+Luxgen+Toyota*: As shown in Table V, Table VI, and Table VII, the matching rates of different ID frames from an ECU on their ID fingerprints were consistently high. This observation indicates that the ID number of an ECU does not impact the performance of our system in detecting malicious frames. To further evaluate the performance of IdentifierIDS under more ECUs, we aggregated ECUs on Buick, Luxgen, and Toyota like the approach in [23], where the maximum number of IDs evaluated per ECU was 3. Some fingerprints, including F4, F8, F12, F18, F25, F31, F32, and F41, were not included in Table VIII, because there were already ID fingerprints in the table that exhibit similar characteristics to them. As shown in Table VIII, despite the increase in the number of senders and IDs, the average accuracy was still over 99%, and the average false positive rate dropped to less than 0.1%. This means that IdentifierIDS can establish accurate fingerprints for IDs and can be adapted to different real in-vehicle networks.

C. Detecting Fabrication Attack and Masquerade Attack

On one side, a given ECU can be compromised to act as the attacker ECU, while on the other side, it may also act as the victim ECU after receiving malicious frames. After modifying

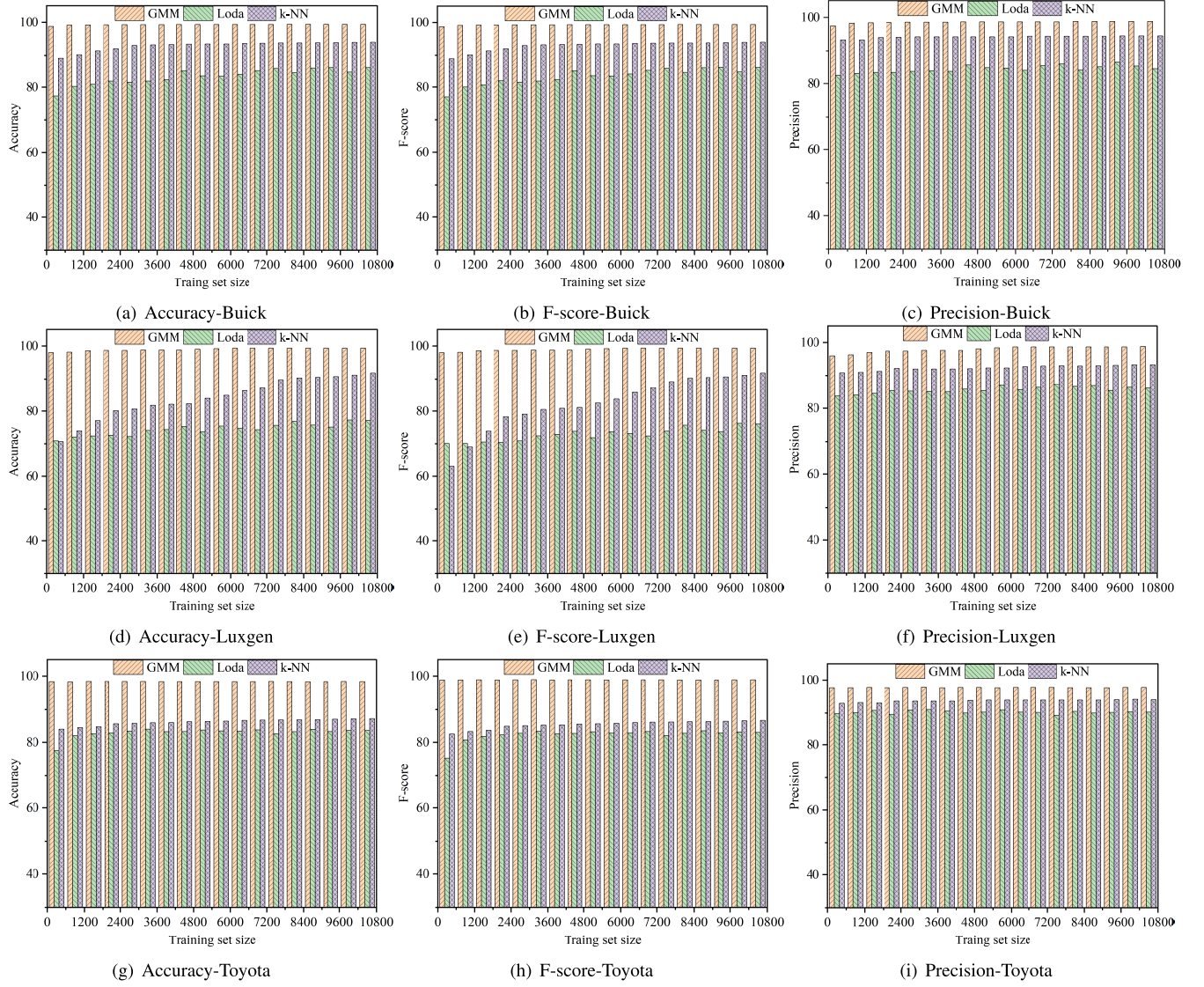


Fig. 7. Average detection accuracy, F-score, and precision on Buick, Luxgen, and Toyota.

the ID of frames, normal frames can be masqueraded as or fabricated as originating from a given ECU. Based on this, we modified the ID of test frames in turn, thereby obtaining malicious frames originating from the compromised ECU.

As shown in Fig. 7, we evaluated the performance of IdentifierIDS under different training set sizes and different algorithms. The size of the initial training set was 300, which was increased by 600 samples each time up to 10800 samples. The lightweight on-line detector of anomalies (Loda), which is a novel machine learning algorithm proposed in 2016, can detect anomalies quickly and accurately [44]. The k nearest neighbor method (k-NN), which is a classic supervised learning algorithm proposed in [45], uses the distance to the k th nearest neighbor as the outlier score to detect outliers. We employed these two anomaly detection algorithms for a comparative analysis with the GMM algorithm, aiming to determine the more appropriate algorithm for IdentifierIDS. Our evaluation metrics include accuracy, F-score, and precision.

The number of correctly identified legitimate signals by IdentifierIDS is denoted as TP , while the number of legitimate

signals incorrectly identified is denoted as FN . The quantity of malicious signals originating from the compromised ECU detected by IdentifierIDS is represented as TN , while the quantity that went undetected is indicated as FP . *Accuracy* represents the proportion of legitimate and malicious signals correctly identified by IdentifierIDS. *Precision* represents the proportion of signals identified as legitimate by IdentifierIDS that are correctly identified, while *recall* represents the proportion of legitimate signals that are correctly identified. *F-score*, which is equivalent to the harmonic mean of *precision* and *recall*, can comprehensively evaluate the system. The formulas of the three indicators are as follows:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$F-score = \frac{2TP}{2TP + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

Fig. 7 sequentially shows the performance of GMM, Loda, and k-NN in detecting frames collected from

TABLE IX

THE MATCHING RATE OF MALICIOUS FRAMES FROM ADDITIONAL NODES ON THE FINGERPRINTS OF THREE VEHICLES

Vehicle	Recognition rate													
Buick	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10	F11	F12	F13	F14
	100	100	100	100	100	100	100	100	100	100	100	100	100	100
Luxgen	F15	F16	F17	F18	F19	F20	F21	F22	F23	F24	F25	F26	F27	
	100	100	100	100	100	100	100	100	100	100	100	100	100	
Toyota	F28	F29	F30	F31	F32	F33	F34	F35	F36	F37	F38	F39	F40	F41
	100	100	100	100	100	100	100	100	100	100	100	100	100	100

three real vehicles (including Buick, Luxgen, and Toyota). Figs. 7(a), 7(d), and 7(g) are the average accuracy of all models on frames from each vehicle, respectively. Figs. 7(b), 7(e), and 7(h) represent the average F-score of all models on frames from each vehicle, respectively. Figs. 7(c), 7(f), and 7(i) are the average precision on frames from each vehicle. It is evident that as the training set size increases, all three indicators increase to some extent until they reach the maximum value. In addition, the GMM algorithm performed better than the Loda algorithm and k-NN algorithm on training sets of any size on three vehicles. We note that among the three metrics, the precision varies the least with the increase in the number of training samples. This is because we set the threshold as the mean of the training sample scores multiplied by a specific coefficient. Consequently, precision remains relatively stable as long as the coefficients remain fixed. Moreover, the expansion in the number of training samples exerts a minor influence on the training sample scores. Finally, after the number of training samples reached 6900, the model's accuracy, F-score, and precision achieved the maximum.

D. Detecting Additional Attack

In this section, we evaluated the performance of IdentifierIDS in detecting malicious frames from additional nodes. As we mentioned in Subsection II-B, frames from different ECUs have different voltage characteristics. Therefore, malicious frames from additional nodes are abnormal data for the CAN bus. A signal that does not match all fingerprints in IdentifierIDS can be considered from an additional node.

As shown in Table IX, we evaluated the ability of IdentifierIDS to detect malicious frames from additional nodes. F1 to F14 are ID fingerprints belonging to Buick, F15 to F27 are ID fingerprints from Luxgen, and F28 to F41 are ID fingerprints belonging to Toyota. It is worth noting that the fingerprints used to detect fabrication attack and masquerade attack can still be used to detect additional nodes. If a frame from an additional node matches any ID fingerprint, it can be considered as bypassing IdentifierIDS. This is because no additional node should be allowed to use any recorded ID. As shown in Table IX, all malicious frame does not match any fingerprint, which means all fingerprints of the three vehicles can identify all malicious frames. Therefore, the identification rates of IdentifierIDS against additional attacks on three vehicles are 100%. It is unlikely that the signal from the additional node is so similar to the signal from the ECU inside the vehicle that it would produce the false positive, but

TABLE X

THE CONFUSION MATRIX FOR DETECTING BUS-OFF ATTACK

	True	Predicted		FPR
		Bus-off attack	Normal	
Buick	Bus-off attack	100	0	0
	Normal	0.05	99.95	
Luxgen	Bus-off attack	100	0	0
	Normal	0.03	99.97	
Toyota	Bus-off attack	100	0	0
	Normal	0.1	99.9	

we cannot rule out the possibility. Fortunately, the varying cable lengths and wiring resistances among different ECUs or nodes further contribute to reducing this potential [25].

E. Detecting Bus-off Attack

To validate the detection capability of IdentifierIDS for bus-off attack, we collected 4000 corrupted signals and 4000 normal signals from each vehicle. Each corrupted signal comprises a dominant bit superimposed by two ECUs and two unaffected edges.

Table X shows the confusion matrix for detecting bus-off. Due to the significant difference between the superimposed dominant bit and the normal dominant bit, the collected signals, despite containing two normal edges, are sufficiently distinguishable for ID fingerprint recognition. As a result, the identification rates for bus-off attack were 100% across all three vehicles. The false positive rate (FPR) represents the proportion of bus-off attack frames that are misidentified as normal frames. IdentifierIDS had an FPR of 0 in all three vehicles. A normal frame can only be incorrectly identified as originating from a bus-off attack if it fails to match all ID fingerprints. Therefore, across all three vehicles, only a very small number of normal frames failed to match all ID fingerprints.

F. Time Expenditure and Memory Footprint

Since the promptness of IDS is critical for ICVs, we evaluated the time performance of IdentifierIDS in training models and matching signals on Raspberry Pi.

Table XI shows training time and signal matching time of fingerprints on three vehicles. We collected 10200 signals for each ID to establish its model (fingerprint). It can be seen that the average time to train a model using GMM and Loda algorithms on three vehicles did not exceed 0.31 s, while the average duration of establishing a fingerprint using k-NN exceeded 1.29 s. Furthermore, the fingerprint trained by the GMM algorithm took 0.00355 ms, 0.00367 ms, and 0.00348 ms to test whether a signal matches the fingerprint in Buick, Luxgen, and Toyota, respectively. However, the model trained by Loda or the k-NN algorithm took more than 4 times longer than the GMM algorithm model to match a signal. Therefore, it is necessary for IdentifierIDS to use the GMM algorithm to identify the sender of the signal quickly. The normal frame only needs to be matched with

TABLE XI
THE TRAINING TIME AND SIGNAL MATCHING TIME OF
FINGERPRINTS ON THREE VEHICLES

Vehicle	Algorithm	Training time (10200 signals)	Matching time (10000 signals)	Matching time (per signal)
Buick	GMM	140.93 ms	35.51 ms	0.003551 ms
	Loda	297.99 ms	160.96 ms	0.016096 ms
	k-NN	1301.86 ms	13233.91 ms	1.323391 ms
Luxgen	GMM	139.54 ms	36.71 ms	0.003671 ms
	Loda	301.29 ms	164.54 ms	0.016454 ms
	k-NN	1398.69 ms	13159.45 ms	1.315945 ms
Toyota	GMM	134.61 ms	34.78 ms	0.003478 ms
	Loda	289.63 ms	156.27 ms	0.015627 ms
	k-NN	1299.48 ms	10921.37 ms	1.092137 ms

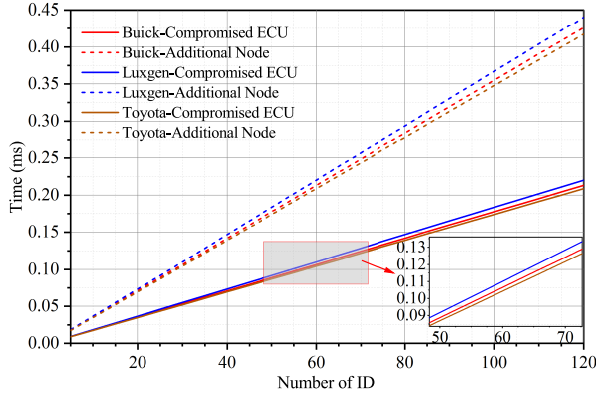


Fig. 8. Matching time as a function of number of IDs.

the fingerprint corresponding to its ID and not with other ID fingerprints. Similarly, a malicious frame only needs to undergo one matching process to be detected. This implies that the time taken to match a malicious frame on the three vehicles were 0.00355 ms, 0.00367 ms, and 0.00348 ms, respectively.

After the malicious frame failed to match the model for the first time, IdentifierIDS would match it with other models, in turn, to find the attack source. Fig. 8 shows the time overhead of our scheme for matching compromised ECU and additional node in scenarios with different numbers of IDs. Since the compromised ECU may be any ECU of in-vehicle network, the average number ($Number_{average}$) of matches on a vehicle with N IDs is as follows:

$$Number_{average} = \frac{\sum_{i=1}^{N-1} i}{N-1} = \frac{N}{2}$$

When the average matching time on a fingerprint is T , the average time to identify the compromised ECU is $T \times \frac{N}{2}$. Since malicious frames from additional nodes would fail to match in all fingerprints, the number of matching times is N , and the time overhead for identifying is $T \times N$. According to the data from 9 real vehicles in research [33], the average number of IDs in CAN bus is 44. In the case of 44 IDs, IdentifierIDS took (0.0781 ms, 0.1562 ms) in Buick, (0.0807 ms, 0.1615 ms) in Luxgen, and (0.0766 ms, 0.1531 ms) in Toyota to find attack source (compromised ECU, additional node). In fact, most of the frames on the CAN bus are normal, which means they only

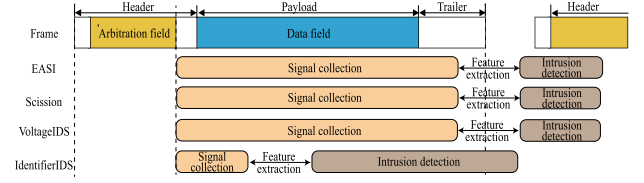


Fig. 9. Comparison of time costs in the signal collection phase among IdentifierIDS and multiple representative schemes.

need to match a fingerprint without locating the compromised ECU and additional node.

Fig. 9 shows the comparison of time costs in the signal collection phase among IdentifierIDS and multiple representative schemes (including EASI [28], Scission [26], and VoltageIDS [25]) during intrusion detection. The time cost of signal collection mainly includes the waiting time for frame transmission and the actual signal collection time. Specifically, frames are transmitted bit by bit onto the CAN bus, and IDSs receive the signals bit by bit as well. To avoid collecting signals affected by arbitration, IDSs wait until the arbitration process is completed before starting the signal collection.

EASI, Scission, and VoltageIDS collect all signals from the IDE bit to the CRC field to establish fingerprints. IdentifierIDS only collects the first rising edge, the first dominant bit, and the first falling edge after the arbitration field. After the ECU sends 5 consecutive bits with the same state, it inserts a bit with the opposite state. Consequently, it can be determined that there is at least a rising edge, a falling edge, and a dominant bit within the IDE bit to the first byte of the data field. Transmitting 1 bit on a 500 kbit/s CAN bus takes $2 \mu s$. This means that when detecting a data frame with an 8-byte data field, IdentifierIDS completes the signal collection $144 \mu s$ ($2 \mu s \times 72$ bit) ahead of the scheme that collects the signal within the IDE bit to the CRC field. Therefore, it can be considered that IdentifierIDS has a lower time overhead of signal collection than these IDSs [25], [26], [28] in signal collection phase.

When establishing ID fingerprints for IdentifierIDS, the memory footprint primarily comprises the memory footprint of the training dataset and the memory footprint of the trained model. To assess this, we employed the `sizeof` function of `sys` library [46] for memory profiling. It is worth noting that IdentifierIDS trains fingerprints for one ID at a time, which means that only the training data for a single ID is held in memory. We observed that the memory footprint for the training data of one ID is 254616 bytes, and the memory footprint for training a model is 48 bytes. The MPC5748G is a microcontroller unit designed for automotive gateways, featuring 768 KB of memory [47], which comfortably accommodates the memory requirements during the fingerprint training process for IdentifierIDS.

V. DISCUSSION

A. Stability

In [33], Popa et al. have shown the variations between the voltage signal collected from a vehicle after cold-starting and the voltage signal collected from the same vehicle after

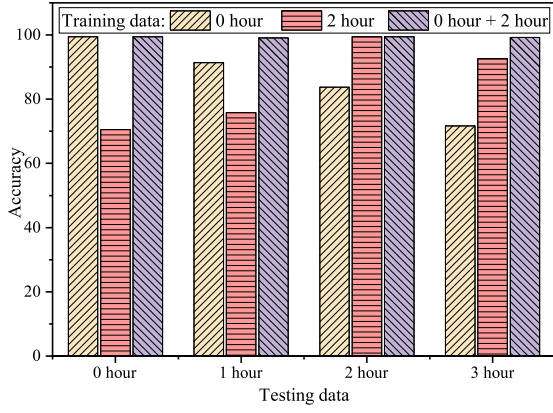


Fig. 10. Performance of IdentifierIDS in detecting signals collected after different time points.

running for one hour. To demonstrate the performance of IdentifierIDS at various stages of vehicle runtime, the recording of voltage signals was started after different time points: after vehicle cold start (0 hour), after 1 hour of runtime, after 2 hours of runtime, and after 3 hours of runtime. These signals were collected from the OBD-II port of the Luxgen. Same as [33], we observed that the mean voltage level collected after a period of runtime exceeded the mean voltage level collected from a vehicle after cold-starting. For instance, the mean dominant-state voltage level of frames with ID 0x316 collected from the vehicle after cold-starting was 2.0511 V, while it increased to 2.0768 V after 2 hours of vehicle runtime.

Fig. 10 shows the performance of IdentifierIDS in detecting signals collected after different time points. We established ID fingerprints using three separate datasets, which consisted of data collected from cold-started (0-hour) vehicle, data collected from vehicle running for 2 hours, and the combination of these two datasets. Fingerprints established by training data collected after one-time points exhibited a certain degree of reduced accuracy when identifying signals collected after different moments. We observed that fingerprints established from the combination of training data could accurately identify signals collected after different time points, including signals collected after 1 hour of runtime and 3 hours of runtime. This means that the IdentifierIDS is capable of establishing robust fingerprints by learning from diverse signals.

Due to the voltage signal being affected by multiple environmental factors, such as the environmental temperature, supply voltage, etc., the characteristics of voltage signals generated by ECUs may vary. It could be worth considering approaches like retraining, utilizing incremental learning to update voltage fingerprints, or collecting diverse data to establish fingerprints, enhancing the robustness of IDS.

B. Locating Compromised ECU

Locating the sender ECU of malicious frames not only helps users remove the compromised ECU in time but also assists vehicle manufacturers in tracing the attack path. This information is valuable for identifying vulnerabilities and implementing appropriate security measures to prevent future attacks. IdentifierIDS can ascertain an ID of the compromised

ECU, but it cannot directly locate the specific physical device associated with that ID. The ID information obtained from IdentifierIDS is still meaningful, since the vehicle manufacturer is aware of the mapping between ECUs and IDs. This information obtained from IdentifierIDS can help vehicle manufacturers identify which physical device is compromised.

C. Device for Deploying IdentifierIDS

IdentifierIDS can be deployed on an external device that can access the CAN bus, as well as on devices that may be introduced in the future to meet the requirements of the scheme. On one hand, for vehicles whose hardware does not meet the requirements of IdentifierIDS, the IDS can be deployed on an external device. Unfortunately, the required sampling rate of 62.5 MS/s for IdentifierIDS is higher than the typical sampling rate of the traditional ECU. It should be noted that there are affordable sampling devices available that meet the requirements of IdentifierIDS, such as the ADC12DL080 [40] from Texas Instruments. Considering that the ADC12DL080 and the Raspberry Pi, which meet the requirements of IdentifierIDS, are priced at under \$65, this does not impose a high cost for external devices.

On the other hand, as vehicles become increasingly intelligent, connected, and electric, vehicle manufacturers are equipping vehicles with increasingly powerful hardware to enable more and more functions. For example, the MPC-LS-VNP-EVB [48] introduced by NXP, which can serve as an advanced gateway, features a frequency that can reach up to 1.6 GHz. This also provides hardware support for the deployment of IDS. Nonetheless, deploying IDS on traditional in-vehicle network ECUs is a worthwhile direction to pursue, as it will enhance the applicability of the IDS. Therefore, we plan to explore a scheme that can be directly deployed on traditional ECUs in future work.

VI. CONCLUSION

We proposed a practical voltage-based IDS named IdentifierIDS for the real in-vehicle network. As the proposed approach establishes a fingerprint for each ID, it can detect intrusions without the prior knowledge of mapping between ECUs and IDs. Compared with clock-based IDS, our established voltage fingerprint can detect malicious frames with aperiodic ID. Our scheme, as a self-learning IDS, can adapt to different in-vehicle networks without the need for customization for different vehicle models. We also provided a series of experimental results that demonstrate the feasibility of IdentifierIDS in detecting multiple attacks.

APPENDIX

Popa et al. [33] demonstrated that only using four simplified voltage features enables the differentiation of signals originating from different ECUs. This scheme collected these features from the complete signal at a sampling rate of 500 MS/s. However, we found that establishing fingerprints for IdentifierIDS using only these features is not well suited.

TABLE XII

MATCHING MATRIX OF SIGNALS COLLECTED USING FOUR SIMPLIFIED FEATURES ON 14 ID FINGERPRINTS OF BUICK

		ECU A				ECU B				ECU C				ECU D	
		0AA	0BE	0C9	0F9	0B9	0BA	0BB	0BC	0C1	0C5	0D1	1C6	0F1	1E1
ECU A	F1 (0AA)	99.5	99.0	99.1	99.3	0	0	0	0	88.3	90.6	91.0	90.8	0	0
	F2 (0BE)	99.1	99.0	99.0	99.3	0	0	0	0	90.1	91.4	88.6	90.1	0	0
	F3 (0C9)	98.8	98.4	98.5	99.1	0	0	0	0	94.2	89.4	90.5	89.8	0	0
	F4 (0F9)	98.1	98.3	98.6	98.1	0	0	0	0	89.2	91.4	92.3	88.4	0	0
ECU B	F5 (0B9)	0	0	0	0	99.0	99.8	99.7	99.8	0	0	0	0	0	0
	F6 (0BA)	0	0	0	0	97.5	99.0	98.6	98.4	0	0	0	0	0	0
	F7 (0BB)	0	0	0	0	99.0	99.8	99.5	99.8	0	0	0	0	0	0
	F8 (0BC)	0	0	0	0	98.8	99.7	98.4	99.0	0	0	0	0	0	0
ECU C	F9 (0C1)	83.2	82.0	80.3	78.5	0	0	0	0	98.4	99.0	99.1	99.0	0	0
	F10 (0C5)	84.4	83.9	81.3	79.2	0	0	0	0	99.8	99.8	99.9	99.9	0	0
	F11 (0D1)	82.6	83.3	80.7	81.0	0	0	0	0	99.8	99.7	98.5	99.8	0	0
	F12 (1C6)	84.1	83.4	80.4	78.0	0	0	0	0	99.8	99.7	99.2	99.0	0	0
ECU D	F13 (0F1)	0	0	0	0	0	0	0	0	0	0	0	0	99.1	98.5
	F14 (1E1)	0	0	0	0	0	0	0	0	0	0	0	0	98.6	99.1

To demonstrate the performance of utilizing these features in our scenario with sampling rate of 62.5 MS/s for distinguishing different ECU signals, we replaced the features used by IdentifierIDS with these four features. Subsequently, we established voltage fingerprints for 14 IDs of Buick. The matching results of frames on these ID fingerprints are shown in Table XII. Each ID fingerprint can accurately match the frames from its corresponding ECU. However, we observed that the frames from ECU A have a high matching rate with the fingerprints F9, F10, F11, and F12 corresponding to ECU C. This indicates that the fingerprints established using four features from the scheme (so-called ECUPrint [33]) were not effective in distinguishing frames of ECU A from frames of ECU C. In contrast, as shown in Table V, the ID fingerprints established using our features can effectively differentiate the frames from these four ECUs. It is not surprising that these features perform poorly in distinguishing the ECUs in Buick, as IdentifierIDS uses a much lower sampling rate of 62.5 MS/s compared to the 500 MS/s used in ECUPrint. Generally, higher sampling rate results in better performance of IDS [26]. It should be noted that we only utilized these four features for training the GMM fingerprints and did not employ the threshold-based approach outlined in ECUPrint [33] to distinguish message senders. This may also account for the difference in experimental results between this paper and ECUPrint.

REFERENCES

- [1] C. Miller and C. Valasek, "Remote exploitation of an unaltered passenger vehicle," *Black Hat USA*, vol. 2015, no. S91, pp. 1–91, 2015.
- [2] S. Nie, L. Liu, Y. Du, and W. Zhang, "Over-the-air: How we remotely compromised the gateway, BCM, and autopilot ECUs of Tesla cars," *Briefing, Black Hat USA*, vol. 91, pp. 1–19, Aug. 2018.
- [3] T. K. Lab. (2021). *Mercedes-Benz MBUX Security Research Report*. Accessed Jul. 3, 2023. [Online]. Available: https://keenlab.tencent.com/en/whitepapers/Mercedes-Benz_Security_Research_Report_Final.pdf
- [4] S. Woo, H. J. Jo, and D. H. Lee, "A practical wireless attack on the connected car and security protocol for in-vehicle CAN," *IEEE Trans. Intell. Transp. Syst.*, vol. 16, no. 2, pp. 993–1006, Apr. 2015.
- [5] D. K. Nilsson, U. E. Larson, and E. Jonsson, "Efficient in-vehicle delayed data authentication based on compound message authentication codes," in *Proc. IEEE 68th Veh. Technol. Conf.*, Sep. 2008, pp. 1–5.
- [6] Z. Gu, G. Han, H. Zeng, and Q. Zhao, "Security-aware mapping and scheduling with hardware co-processors for FlexRay-based distributed embedded systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 27, no. 10, pp. 3044–3057, Oct. 2016.
- [7] S. Jin, J.-G. Chung, and Y. Xu, "Signature-based intrusion detection system (IDS) for in-vehicle CAN bus network," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2021, pp. 1–5.
- [8] H. J. Jo, J. H. Kim, H.-Y. Choi, W. Choi, D. H. Lee, and I. Lee, "MAuth-CAN: Masquerade-attack-proof authentication for in-vehicle networks," *IEEE Trans. Veh. Technol.*, vol. 69, no. 2, pp. 2204–2218, Feb. 2020.
- [9] R. Bhatia, V. Kumar, K. Serag, Z. B. Celik, M. Payer, and D. Xu, "Evading voltage-based intrusion detection on automotive CAN," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2021, pp. 1–17.
- [10] H. M. Song, H. R. Kim, and H. K. Kim, "Intrusion detection system based on the analysis of time intervals of CAN messages for in-vehicle network," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Jan. 2016, pp. 63–68.
- [11] A. Taylor, N. Japkowicz, and S. Leblanc, "Frequency-based anomaly detection for the automotive CAN bus," in *Proc. World Congr. Ind. Control Syst. Secur. (WCICSS)*, Dec. 2015, pp. 45–49.
- [12] M. Muter, A. Groll, and F. C. Freiling, "A structured approach to anomaly detection for in-vehicle networks," in *Proc. 6th Int. Conf. Inf. Assurance Secur.*, Aug. 2010, pp. 92–98.
- [13] A. R. Javed, S. u. Rehman, M. U. Khan, M. Alazab, and G. T. Reddy, "CANintelliIDS: Detecting in-vehicle intrusion attacks on a controller area network using CNN and attention-based GRU," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 1456–1466, Apr./Jun. 2021.
- [14] H. Lee, S. H. Jeong, and H. K. Kim, "OTIDS: A novel intrusion detection system for in-vehicle network by using remote frame," in *Proc. 15th Annu. Conf. Privacy, Secur. Trust (PST)*, Aug. 2017, pp. 57–5709.
- [15] M. Muter and N. Asaj, "Entropy-based anomaly detection for in-vehicle networks," in *Proc. IEEE Intell. Vehicles Symp. (IV)*, Jun. 2011, pp. 1110–1115.
- [16] M. Marchetti, D. Stabili, A. Guido, and M. Colajanni, "Evaluation of anomaly detection for in-vehicle networks through information-theoretic algorithms," in *Proc. IEEE 2nd Int. Forum Res. Technol. Soc. Ind. Leveraging Better Tomorrow (RTSI)*, Sep. 2016, pp. 1–6.
- [17] D. Stabili, M. Marchetti, and M. Colajanni, "Detecting attacks to internal vehicle networks through Hamming distance," in *Proc. AEIT Int. Annu. Conf.*, Sep. 2017, pp. 1–6.
- [18] K. Zheng, S. Zou, G. Xu, and Z. Bi, "Segment detection algorithm: CAN bus intrusion detection based on bit constraint," in *Proc. IEEE 23rd Int. Symp. a World Wireless, Mobile Multimedia Netw. (WoWMoM)*, Jun. 2022, pp. 450–456.
- [19] B. Groza and P.-S. Murvay, "Efficient intrusion detection with Bloom filtering in controller area networks," *IEEE Trans. Inf. Forensics Security*, vol. 14, no. 4, pp. 1037–1051, Apr. 2019.
- [20] K.-T. Cho and K. G. Shin, "Fingerprinting electronic control units for vehicle intrusion detection," in *Proc. 25th USENIX Secur. Symp. (USENIX Security)*, 2016, pp. 911–927.
- [21] S. Kulandaivel, T. Goyal, A. K. Agrawal, and V. Sekar, "CANvas: Fast and inexpensive automotive network mapping," in *Proc. 28th USENIX Secur. Symp. (USENIX Security)*, 2019, pp. 389–405.
- [22] Y. Zhao, Y. Xun, and J. Liu, "ClockIDS: A real-time vehicle intrusion detection system based on clock skew," *IEEE Internet Things J.*, vol. 9, no. 17, pp. 15593–15606, Sep. 2022.
- [23] K.-T. Cho and K. G. Shin, "Viden: Attacker identification on in-vehicle networks," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2017, pp. 1109–1123.
- [24] W. Choi, H. J. Jo, S. Woo, J. Y. Chun, J. Park, and D. H. Lee, "Identifying ECUS using inimitable characteristics of signals in controller area networks," *IEEE Trans. Veh. Technol.*, vol. 67, no. 6, pp. 4757–4770, Jun. 2018.
- [25] W. Choi, K. Joo, H. J. Jo, M. C. Park, and D. H. Lee, "VoltageIDS: Low-level communication characteristics for automotive intrusion detection system," *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 8, pp. 2114–2129, Aug. 2018.
- [26] M. Kneib and C. Huth, "Scission: Signal characteristic-based sender identification and intrusion detection in automotive networks," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2018, pp. 787–800.
- [27] M. Foruhandeh, Y. Man, R. Gerdes, M. Li, and T. Chantem, "SIMPLE: Single-frame based physical layer identification for intrusion detection and prevention on in-vehicle networks," in *Proc. 35th Annu. Comput. Secur. Appl. Conf.*, Dec. 2019, pp. 229–244.

- [28] M. Kneib, O. Schell, and C. Huth, "EASI: Edge-based sender identification on resource-constrained platforms for automotive networks," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020, pp. 1–16.
- [29] P.-S. Murvay and B. Groza, "Source identification using signal characteristics in controller area networks," *IEEE Signal Process. Lett.*, vol. 21, no. 4, pp. 395–399, Apr. 2014.
- [30] Y. Xun, Y. Zhao, and J. Liu, "VehicleEIDS: A novel external intrusion detection system based on vehicle voltage signals," *IEEE Internet Things J.*, vol. 9, no. 3, pp. 2124–2133, Feb. 2022.
- [31] N. Liu, C. Moreno, M. Dunne, and S. Fischmeister, "vProfile: Voltage-based anomaly detection in controller area networks," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Feb. 2021, pp. 1142–1147.
- [32] Z. Deng, Y. Xun, J. Liu, S. Li, and Y. Zhao, "A novel intrusion detection system for next generation in-vehicle networks," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Dec. 2022, pp. 2098–2103.
- [33] L. Popa, B. Groza, C. Jichici, and P.-S. Murvay, "ECUPrint—Physical fingerprinting electronic control units on CAN buses inside cars and SAE J1939 compliant vehicles," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 1185–1200, 2022.
- [34] K.-T. Cho and K. G. Shin, "Error handling of in-vehicle networks makes them vulnerable," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2016, pp. 1044–1055.
- [35] S. Lee, W. Choi, and D. H. Lee, "Usable user authentication on a smartwatch using vibration," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 304–319.
- [36] S. Dey, N. Roy, W. Xu, R. R. Choudhury, and S. Nelakuditi, "AccelPrint: Imperfections of accelerometers make smartphones trackable," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2014, pp. 1–6.
- [37] Y. Lei, Z. He, Y. Zi, and Q. Hu, "Fault diagnosis of rotating machinery based on multiple ANFIS combination with GAs," *Mech. Syst. Signal Process.*, vol. 21, no. 5, pp. 2280–2294, Jul. 2007.
- [38] A. Phinyomark, P. Phukpattaranont, and C. Limsakul, "Feature reduction and selection for EMG signal classification," *Expert Syst. Appl.*, vol. 39, no. 8, pp. 7420–7431, Jun. 2012.
- [39] L. H. N. Lorena, A. C. P. L. F. Carvalho, and A. C. Lorena, "Filter feature selection for one-class classification," *J. Intell. Robot. Syst.*, vol. 80, no. S1, pp. 227–243, Dec. 2015.
- [40] *ADC12DL080*. Accessed: Jul. 3, 2023. [Online]. Available: <https://www.ti.com/product/ADC12DL080>
- [41] *Raspberry Pi 4 Tech Specs*. Accessed: Jul. 3, 2023. [Online]. Available: <https://www.raspberrypi.com/news/raspberry-pi-4-on-sale-now-from-35/pyod>
- [42] *pyod*. Accessed: Jul. 3, 2023. [Online]. Available: <https://github.com/yzhao062/pyod>
- [43] *Canvas*. Accessed: Jul. 3, 2023. [Online]. Available: <https://github.com/sekarity/canvas>
- [44] T. Pevný, "Loda: Lightweight on-line detector of anomalies," *Mach. Learn.*, vol. 102, no. 2, pp. 275–304, Feb. 2016.
- [45] S. Ramaswamy, R. Rastogi, and K. Shim, "Efficient algorithms for mining outliers from large data sets," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, May 2000, pp. 427–438.
- [46] *sys*. Accessed: Oct. 7, 2023. [Online]. Available: <https://docs.python.org/3/library/sys.html>
- [47] *Future Advances in Body Electronics*. Accessed: Oct. 7, 2023. [Online]. Available: https://www.nxp.com/docs/en/white-paper/BODY_DELECTRWP.pdf
- [48] *MPC-LS Vehicle Network Processing Evaluation Board*. Accessed: Oct. 7, 2023. [Online]. Available: <https://www.nxp.com/design/development-boards/automotive-development-platforms/mpc57xx-mcu-platforms/mpc-ls-vehicle-network-processing-evaluation-board:MPC-LS-VNP-EVB>



Jiajia Liu (Senior Member, IEEE) is currently a Full Professor (Vice Dean) with the School of Cybersecurity, Northwestern Polytechnical University. He has published more than 180 peer-reviewed papers in many high quality publications, including prestigious IEEE journals and conferences. His research interests include intelligent and connected vehicles, mobile/edge/cloud computing and storage, the Internet of Things security, wireless and mobile ad hoc networks, and space-air-ground integrated networks. He received the IEEE VTS Early Career Award in 2019, the IEEE ComSoc Asia-Pacific Outstanding Young Researcher Award in 2017, the IEEE ComSoc Asia-Pacific Outstanding Paper Award in 2019, the Niwa Yasujiro Outstanding Paper Award in 2012, the best paper awards from many international conferences, including IEEE flagship events, such as the IEEE GLOBECOM in 2016 and 2019, the IEEE WCNC in 2012 and 2014, the IEEE WiMob in 2019, and the IEEE IC-NIDC in 2018. He was a recipient of the Tohoku University President Award 2013. He is serving as technical program committees for numerous international conferences, such as the leading Symposium Co-Chair for the AHSN symposium for GLOBECOM 2017, the CRN symposium for ICC 2018, and the AHSN symposium for ICC 2019. He is the Vice Chair of the IEEE IOT-AHSN TC and a Distinguished Lecturer of IEEE Communications Society and Vehicular Technology Society. He has been actively joining the society activities, such as serving as an Associate Editor for IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS since May 2018, IEEE TRANSACTIONS ON COMPUTERS from October 2015 to June 2017, and IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY since January 2016; an Editor for IEEE NETWORK since July 2015 and the IEEE TRANSACTIONS ON COGNITIVE COMMUNICATIONS AND NETWORKING since January 2019; and the Guest Editor for top ranking international journals, such as IEEE TRANSACTIONS ON EMERGING TOPICS IN COMPUTING, *IEEE Vehicular Technology Magazine*, and *IEEE INTERNET OF THINGS JOURNAL*.



Yijie Xun (Member, IEEE) received the B.S. degree from Shanxi University, Taiyuan, China, in 2016, and the Ph.D. degree from Xidian University, Xi'an, China, in 2021. He is currently an Associate Professor with the School of Cybersecurity, Northwestern Polytechnical University, Xi'an. His research interests include vehicular network security and machine learning.



Zhouyan Deng (Member, IEEE) received the B.S. degree from the Chengdu University of Information Technology in 2020 and the master's degree from Xidian University in 2023. He is currently pursuing the Ph.D. degree with the School of Cybersecurity, Northwestern Polytechnical University. His research interests include vehicular network security and machine learning.



Junman Qin (Graduate Student Member, IEEE) received the B.S. degree in computer science and technology from Anhui Normal University in 2019 and the master's degree in cyberspace security from Xidian University in 2022. She is currently pursuing the Ph.D. degree with the School of Cybersecurity, Northwestern Polytechnical University. Her research interests include vehicular network security and intrusion detection systems.